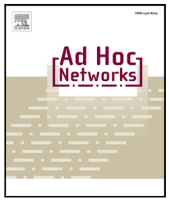
Ad Hoc Networks 157 (2024) 103462



Contents lists available at ScienceDirect

Ad Hoc Networks

journal homepage: www.elsevier.com/locate/adhoc





Adaptive client selection with personalization for communication efficient Federated Learning

Allan M. de Souza [a,*], Filipe Maciel [a,d], Joahannes B.D. da Costa [a], Luiz F. Bittencourt [a], Eduardo Cerqueira [b], Antonio A.F. Loureiro [c], Leandro A. Villas [a]

[a] Universidade Estadual de Campinas, Campinas, Brazil
[b] Federal University of Pará, Belém, Brazil
[c] Federal University of Minas Gerais, Belo Horizonte, Brazil
[d] Federal University of Ceará, Russas, Brazil

ARTICLE INFO

Keywords:
Federated Learning
Client selection
Personalization
Communication efficient

ABSTRACT

Federated Learning (FL) is a distributed approach to collaboratively training machine learning models. FL requires a high level of communication between the devices and a central server, thus imposing several challenges, including communication bottlenecks and network scalability. This article introduces ACSP-FL, a solution to reduce the overall communication and computation costs for training a model in FL environments. ACSP-FL employs a client selection strategy that dynamically adapts the number of devices training the model and the number of rounds required to achieve convergence. Moreover, ACSP-FL enables model personalization to improve clients performance. A use case based on human activity recognition datasets aims to show the impact and benefits of ACSP-FL when compared to state-of-the-art approaches. Experimental evaluations show that ACSP-FL minimizes the overall communication and computation overheads to train a model and converges the system efficiently. In particular, ACSP-FL reduces communication up to 95% compared to literature approaches while providing good convergence even in scenarios where data is distributed differently, non-independent and identical way between client devices.

1. Introduction

Advances in distributed computing, such as the popularization of Edge Computing, Fog Computing, and, in a more general sense, Mobile Edge Computing (MEC) [1,2], have paved the way for distributed Machine Learning (ML) model training, utilizing the resources of end devices (i.e., devices located at the network edge) [3]. One of the techniques that leverages edge computing for distributed model training is called Federated Learning (FL) [3,4].

FL allows distributed model training, where it enables devices (i.e., clients) to fit a global model in their local data and then shares the trained model with a central server responsible for aggregating received models (i.e., aggregator). Finally, the model is updated and shared again with clients for retraining. This whole procedure is known as communication round, repeating until the global model reaches a convergence. In other words, FL allows distributed devices to train collaboratively a shared model by keeping all training data on the device and sharing only the locally trained model. Consequently, it is possible to mitigate many systemic privacy risks and costs resulting from traditional, centralized machine learning approaches [5].

The model sharing among clients and the server in FL solutions can introduce additional overhead to the system [6]. Therefore, they must consider the communication cost as an essential factor in the system design. Some variables influence the federated learning communication overhead, including: *(i)* the size of the shared model; *(ii)* the number of communication rounds to achieve model convergence; *(iii)* the number of clients selected per communication round; and *(iv)* the proportion of the shared model ranging from full or partial model sharing (i.e., few layers or subset of parameters). In this scenario, a still open research question is the following: how to reduce communication overhead without degrading the efficiency of the trained model?

The main approaches for reducing communication overhead focus on client selection and compression of the shared model [6]. However, these approaches must be more adaptive to explore the heterogeneity of devices and changes in the context of the environment. For example, *Power-Of-Choice Selection* [7] defines a fixed number of clients for all

* Corresponding author.
E-mail addresses: allanms@ic.unicamp.br (A.M. de Souza), filipe.mr@ufc.br (F. Maciel), joahannes.costa@ic.unicamp.br (J.D. da Costa), bit@unicamp.br (L.F. Bittencourt), cerqueira@ufpa.br (E. Cerqueira), loureiro@dcc.ufmg.br (A.A.F. Loureiro), lvillas@unicamp.br (L.A. Villas).

https://doi.org/10.1016/j.adhoc.2024.103462
Received 24 November 2023; Received in revised form 30 January 2024; Accepted 27 February 2024
Available online 29 February 2024
1570-8705/© 2024 Published by Elsevier B.V.

communication rounds. At the same time, compression solutions use the same settings for all clients during all communication rounds. Other personalization schemes use a local model and a shared one in each client to customize and fine-tune the client model [8]. Therefore, such approach enables partial model sharing, reducing the number of parameters transferred between clients and the server. However, as only some pieces of the model are shared, the client selection should be efficient to allow the system convergence accordingly.

This article presents ACSP-FL (**A**daptive **C**lient **S**election with **P**ersonalization for communication efficient **F**ederated **L**earning) to allow an efficient networking communication procedure in FL environments. ACSP-FL selects clients adaptively based on the global model performance to accelerate the model convergence while reducing communication and computation overheads during the training phase. Furthermore, ACSP-FL reduces the size of the shared model in the uplink (i.e., from clients to the server) and the downlink (i.e., from server to clients) based on a partial model sharing. Finally, to not degrade model performance, ACSP-FL also implements a personalization method, empowering clients to keep high performance even sharing only some layers of the model. The performance results show that ACSP-FL reduces the communication overhead by up to 90% compared to existing solutions in the literature.

This article extends our previous work [9] by introducing new methods to improve both communication and computation aspects in FL environments, where it includes: *(i)* a partial model sharing between a set of clients and a server; and *(ii)* a personalization method to build and fine-tune the model in each client considering local and global models. In addition, this article also presents a detailed description of ACSP-FL, including algorithms, performance evaluation environments, and the behavior and impact of the proposed solution when analyzed with different datasets. The main contributions of this article can be summarized as follows:

- An efficient adaptive client selection approach for FL environments to reduce the communication overhead.
- A personalization mechanism combined with partial model training to reduce the communication overhead without degrading the model's performance.
- The description and implementation of an FL environment[1] (based on containers) for conducting experiments, separating client and server applications into different virtual machines. This division allows the creation of more heterogeneous, manageable, and reproducible environments for experimentation.
- The performance evaluation and assessed metrics to evaluate the proposed solution with literature approaches considering three different datasets. The results outline the benefits of an adaptive client selection scheme with personalization support to enhance the communication process in FL environments compared to other state-of-the-art approaches.

The remainder of this article is organized as follows. Section 2 presents related works, highlighting their advantages and limitations. Section 3 describes the proposed solution, including the methods for adaptive client selection, personalization and partial model sharing. Section 4 discusses the performance evaluation and results obtained. Finally, Section 5 concludes the work and presents some final remarks.

2. Related work

This section describes existing solutions that deal with communication overhead in FL environments, considering the following techniques: *(i)* model compression and sparsification; *(ii)* client selection; and *(iii)* personalization and partial model sharing.

[1] https://github.com/AllanMSouza/ACSP-FL

The Federated Averaging (FedAvg) [10] proposal reduced the number of communication rounds between clients and the central server by exploiting client processing in model training. Clients use several epochs, up to 100, to update the local model before sending it to the server, consequently increasing the system's latency. However, it is not suitable for scenarios non-IID scenarios.

One way to reduce the communication overhead in FL solutions is reducing the representation of the shared model with compression techniques [11], including quantization [12–14] and sparsification [11,15,16], in which a subset of parameters [17]. Ström [16] presents an approach that only sends parameters based on a threshold. Shah and Lau [11] proposes a framework for developing compact models, where each client trains a sparse model locally and then communicates it to the server. Nevertheless, the aggregated model might no longer be sparse after federated averaging. Then, the central server performs a sparse reconstruction of the global model to recover some sparsity loss. There are two advantages to this operation: *(i)* it decreases the volume of downstream communication from the server to the clients, and *(ii)* it allows clients to train a sparse model instead of a dense one, reducing the processing overhead.

On the other hand, Amiri et al. showed that quantization techniques increase communication efficiency [13]. Thus, the central server uses the *Lossy FL* (LFL) algorithm to transmit the quantized global model to clients, reducing the communication overhead without a performance decrease. Bernstein et al. created SignSGD that only transmits the signal of each mini-batch stochastic gradient to alleviate the gradient communication bottleneck problem [14]. However, SignSGD does not perform well in scenarios with non-IID data.

Another way to reduce the communication overhead is the client selection. It selects a subset of available clients to train the shared model in a communication round. Consequently, this reduced participation diminishes the overall communication overhead. Such an approach also proved helpful in accelerating model convergence, reducing the number of rounds required for training the shared model [6,18].

McMahan et al. [10] introduced a random sampling client selection with FedAvg. However, the literature showed that by applying other methods to sampling clients, it is possible to improve a model performance and communication overhead [6]. Nishio and Yonetani [19] observed the importance of systemic aspects, notably the training delay, for prioritizing clients for selection. Goetz et al. [20] already considered statistical metrics about client training to guide the selection. Kang et al. [21] merged both views to classify clients in terms of reputation and thus direct the selection process for the next round of training.

Cho et al. [7] proposed Power-Of-Choice (POC), a computationally and communication-efficient client selection framework that directs the process towards clients with the highest local loss. This selection allows faster convergence, increasing communication efficiency in heterogeneous FL environments. Similarly, the proposal called Oort, created by Lai et al. [22], utilizes loss to determine every client's training utility. However, they added training delay as a weight to consider and designed a more scalable selection algorithm than POC. Both algorithms define a fixed number of clients (i.e., k) to train the global model in all rounds. This lack of adaptation according to the model's performance can provide redundant selections and even unnecessary even after model convergence.

Tan et al. [8] showed that there is poor convergence in scenarios with high data heterogeneity due to the problem of client drift, in which local models converge to a local optimum, degrading the average global model, which does not generalize to clients whose data distribution is quite different from the global one. For these scenarios, a proposed solution is to personalize the models for each client and thus achieve the desired performance. The literature describes two strategies: personalization of the global model and learning of personalized models. Personalizing the global model requires additional training for each client so that the global model adapts to the local data distribution.

Table 1
Description of related works considering client selection, communication, and personalization methods.

Related work	Client selection						Communication			Personalization		
	Random	Statistical	Systemic	Performance	Static	Dynamic	Quantization	Sparsification	Partial Model	Fine Tuning	Layer Sharing	Local Model
McMahan et al. [10]	✓				✓							
Lin et al. [12]							✓					
Amiri et al. [13]							✓					
Bernstein et al. [14]							✓					
Shah and Lau [11]								✓				
Sattler et al. [15]								✓				
Ström [16]								✓				
Nishio and Yonetani [19]			✓		✓							
Goetz et al. [20]		✓			✓							
Kang et al. [21]		✓	✓		✓							
Cho et al. [7]		✓		✓	✓							
Mohamed et al. [26]		✓		✓	✓			✓				
Tan et al. [8]					✓					✓		
de Souza et al. [9]		✓		✓		✓						
Maciel et al. [27]		✓	✓	✓	✓							
Capanema et al. [28]					✓					✓		✓
Lai et al. [22]		✓	✓	✓	✓							
ACSP-FL		✓		✓		✓			✓	✓		

Personalized model learning modifies the aggregation process to allow each client to train a model that suits them.

Several scenarios characterized as non-independent and identically distributed use personalization with success [8]. For example, Chen et al. [23] developed a method where each client can automatically balance the training of its local and global models, implicitly contributing to the training of other clients. Its Self-FL solution was applied to several datasets and non-iid scenarios, demonstrating superior performance to the baseline solutions.

Another fundamental aspect of saving resources is the optimization of the hyperparameters that determine the training behavior. Kairouz et al. [24] identified that one of the challenges for the learning system is identifying the minimum number of participants needed for each round of training for the model to converge. Luo et al. [25] proposed an adaptive federated learning design, where the values of the training control variables are chosen to minimize the total cost and ensure convergence. One of these variables is the minimum number of participants per training round. Although the proposed algorithm determines a configuration that minimizes costs with little work overhead, its hyper-parameterization increases the complexity of the system that uses it.

Considering an adaptive client selection, de Souza et al. [9] introduced DEEV, which samples clients based on their performance and applies a function to dynamically reduce the number of clients selected as the model reaches the convergence. This way, DEEV reduces the communication overhead produced by the federated learning process and also enables a faster model convergence. However, it does not implement any personalization mechanism to ensure a good performance for clients in non-IID scenarios.

Motivated by the solutions presented in the literature and their limitations, we propose ACSP-FL, an approach for adaptive client selection that reduces the communication overhead in federated learning environments while enabling model personalization. Furthermore, ACSP-FL does not define a fixed parameter of clients to be selected and adjusts the number of participants according to the model's performance. Table 1 summarizes the main characteristics of reviewed studies considering system design.

3. ACSP-FL

ACSP-FL[2] aims to reduce the communication and processing overheads generated by FL solutions without degrading the efficiency of machine learning models. To achieve this, ACSP-FL performs an adaptive selection of clients according to the model's performance to decide which devices should perform training in the next round of the communication process, consequently allowing the system to reduce its *overhead* and increase its performance. Moreover, ACSP-FL also implements a personalization mechanism to improve the performance of the models in each client. Lastly, to reduce further the communication cost, ACSP-FL shares only selected layers of the global model to enable collaborative training cost-effectively.

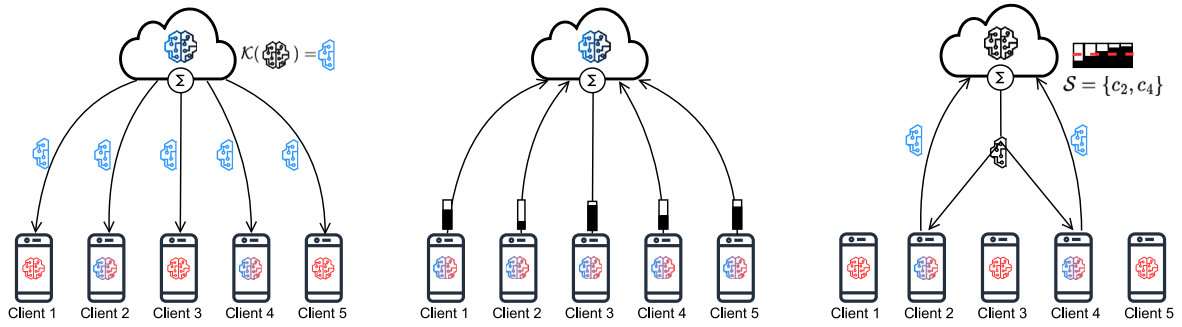
3.1. Problem definition

Let $|C|$ be the number of clients in a scenario where C represents the set of clients, and T is the number of communication rounds. In each round $t \in \{1, 2, 3, \ldots, T\}$, a selected subset $S \subseteq C$ of clients takes part in training the model. Each client $i \in S$ has its local dataset d_i, and $\mathcal{D} = \bigcup_{i \in C} d_i$. Thus, each client receives the current model parameters (i.e., weights) $w(t)$ from the server and updates its local model to train on its data. Subsequently, each client i sends the newly trained model parameters back to the server after training $w^i(t+1)$. The server then aggregates the received models by performing a weighted average, which produces the updated model parameters for the next communication round. At the end of round t, the updated global $w(t+1)$ is computed by averaging the trained model $w^i(t+1)$ of each client i, according to Eq. (1).

$$w^{(t+1)} = \sum_{i \in C} \frac{d_i}{|D|} w_i^{(t)} \tag{1}$$

Hence, the objective is to minimize Eq. (2), where w is the global model, $\mathcal{L}$ represents the model's loss function, and x_i and y_i are the

[2] https://github.com/AllanMSouza/ACSP-FL



(a) Personalization phase (b) Evaluation phase (c) Selection phase

Fig. 1. Performance-based client selection with personalization: personalization phase (a) the server sends the model to the clients, the portion of the model is defined by $\mathcal{K}(\cdot)$; in evaluation phase (b) clients combine the layers shared by the server with their local model to train it with their local data and send back the performance metric to the server; selection phase (c), the server classifies clients according to performance and selects clients with lower performance than the average of received accuracies $\mathcal{A}$ to train the model in the next round of communication.

input and label datasets, respectively.

$$\min f(w) \text{ where } f(w) = \sum_{i=1}^{|C|} \frac{d_i}{|D|} \mathcal{L}(w; x_i, y_i) \tag{2}$$

FedAvg [10] solves Eq. (2) by dividing the training into communication rounds and randomly selecting a set of clients $S \subseteq C$ to train the model. Each client trains the model on its local dataset with τ iterations (i.e., epochs) using the Stochastic Gradient Descent algorithm (SGD) [29] and sends the trained model back to the server. This way, the server updates the global model and shares it again with a new set of clients.

Formally, the set of $0 < i \leq |C|$ clients selected by the server is represented by $S^{(t)}$. This is because the clients execute τ local iterations and the set of clients remains constant for each iteration. Therefore, the model update occurs as follows:

$$w^{(t+1)} = \frac{1}{|S|} \sum_{i \in S} (w_i^{(t)} - \eta_t \mathcal{L}(w_i^{(t)}, n_i)) \tag{3}$$

where $w_i^{(t+1)}$ represents the local model of client i at iteration t, η is the learning rate and $\mathcal{L}(w_i^{(t)}, n_i)$ represents the SGD over the data fraction n of client i. Additionally, $w^{(t+1)}$ defines the global model on the server. Although $w^{(t)}$ is only updated every τ iteration, for convergence reasons, we consider a sequence of $w^{(t)}$ that is updated every iteration. More details on the convergence of models with a partial client selection approach are presented in detail in [7].

3.2. Performance-based client selection

Training models by performance-constrained clients allows models to achieve faster convergence and better average performance of federated learning-based solutions [7]. Thus, clients who already have a good performance wait for clients with lower performance to train their models for more epochs and, consequently, their performance reaches the desired levels. In other words, this approach achieves a faster convergence towards the global average.

To do this, in addition to sharing the model, each client must share a metric with the server so that it is possible to measure the model's performance on its local data, such as training loss or even accuracy. Thus, the server is aware of each client's performance and can apply policies to allow the selection process based on the system performance. Such approaches were used to develop ACSP-FL.

Fig. 1 describes the client selection process applied by ACSP-FL. The overall procedure employed by ACSP-FL is organized as: *(i)* personalization phase; *(ii)* evaluation phase; and *(iii)* client selection phase. The personalization phase is described in Fig. 1(a), in which the model aggregated in the last round is sent to the clients for the following training procedure. However, ACSP-FL does not send the entire model to the clients, and it sends just some layers defined by the function $\mathcal{K}(\cdot)$ (detailed described in the next section). Thus, when clients receive the shared piece, they combine it with its local model for personalization. Hereafter starts the evaluate phase, described by Fig. 1(b), where the clients evaluate the model personalized in its test data and report the performance metric to the server. Finally, when the server receives the performance metric of each client, it defines the set of clients that will be selected to train the model in the following communication round based on the adaptive client selection based on performance. In this way, the selected clients receive the shared piece of the model, personalize it with its local model, fit the model in its local data, and port the shared trained piece to the server. Thus, the described sequence repeats until convergence.

In particular, ACSP-FL selects clients considering their accuracy ($\mathcal{A}_i$). The function $\pi(\cdot)$ is applied as a filtering procedure, so each client that fits the filtering is selected to be part of the set S. The filtering mechanism verifies whether the performance of client $i \in C$ is lower or equal to the mean performance given by the following equation:

$$\pi(i, \mathcal{A}) = \begin{cases} i & \text{if } \mathcal{A}_i \leq \mathcal{A}_{mean} \\ \emptyset & otherwise \end{cases} \tag{4}$$

where $\mathcal{A}$ is the set of accuracies of each client. $\mathcal{A}_i$ and $\mathcal{A}_{mean}$ are the accuracy of the client $i \in C$ and the average accuracy, respectively. Then, by applying the filtering function to all clients in the system, ACSP-FL defines the set of selected clients, represented by:

$$S = \bigcup_{i \in C} \pi(i, \mathcal{A}) \tag{5}$$

where $\pi(\cdot)$ is the filtering method that considers the performance of each client $i \in C$. This approach reduces the communication overhead introduced by federated learning, as fewer clients share models each round, enabling faster convergence. Furthermore, it is important to highlight that the selection procedure implemented by ACSP-FL does not require defining the number of clients a priori, as it is dynamically adjusted at each communication round based on the average performance of the customers.

3.3. Adaptive client selection

Adjusting client selection according to its performance is essential to reduce the use of communication and processing resources. An obstacle to this is the fixed number of selected clients (i.e., k-clients) in each round, even after the model convergence, where it could decrease. Another problem is that selection algorithms might include clients that already perform well and do not contribute to improving the model's convergence just to complete the defined number of k-clients, incurring unnecessary communication and processing.

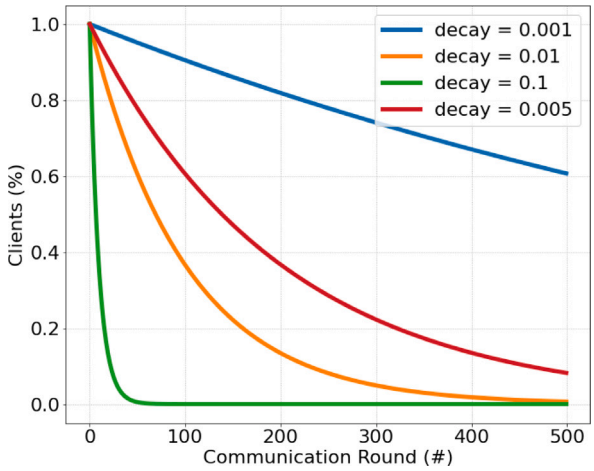


Fig. 2. Example of the decay mechanism employed by ACSP-FL.

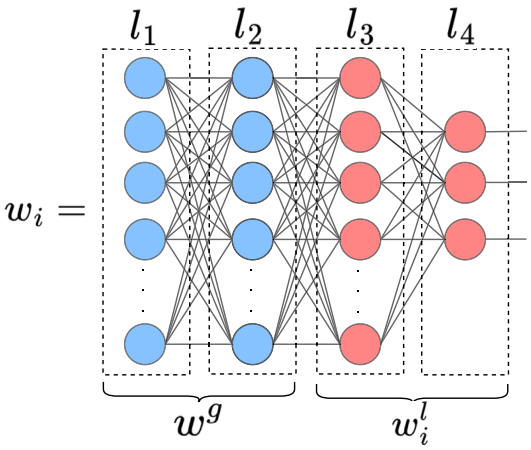


Fig. 3. Model construction employed by ACSP-FL based on the shared and local pieces.

ACSP-FL implements a gradual reduction of clients so that fewer customers are selected in each round based on model's performance. Consequently, further reducing the use of communication and processing resources. The gradual reduction in the number of clients is implemented by a decay function presented in Eq. (6) as follows:

$$\phi(S,t) = \lceil\, |S| \cdot (1 - decay)^t \,\rceil \tag{6}$$

where S represents the set of clients previously selected by ACSP-FL in round t and decay is a parameter to define how gradual the decline in the number of clients will be. Fig. 2 illustrates the gradual decay of clients according to the value of the decay parameter. It is important to highlight that this decay is carried out after defining the adaptive client set implemented by ACSP-FL. As can be seen, different values provide different decays, which must be adjusted to allow a better cost-benefit ratio between performance and overhead.

With the number of clients calculated after defining the set of clients that perform less than the average performance (i.e., $\mathcal{A}_{mean}$, given by Eq. (5)) and also after applying the decay function, ACSP-FL selects the clients as follows:

$$S = \bigcup_{i=0}^{\phi(S,t)} S[i], \text{ such that } \phi(S,t) \leq |S| \tag{7}$$

that is, after ordering the clients' performance ACSP-FL selects the $\phi(S,t)$ first clients with performance below the average.

3.4. Model personalization and layer sharing

Personalization empowers model performance via fine tuning [8]. The idea is to adjust the global model on clients data, thus providing a better fit for each client. The traditional approach for personalization and fine tuning is to keep one local model in each client (i.e., w_i^l $i \in C$, local model of client i) and a global model which is the aggregation of the local models of each client (i.e., $w^g = \sum_{i \in C} \frac{n_i}{N} w_i^l$). Therefore, with those models the client can decide which one will be used based on the one that achieves the best performance, according to the personalization function $\mathcal{P}(w_i^l, w_i^g)$:

$$\mathcal{P}(w_i^l, w^g) = \begin{cases} w_i^l & \mathcal{L}(w_i^l; x_i, y_i) \leq \mathcal{L}(w^g; x_i, y_i) \\ w_i^g & otherwise \end{cases} \tag{8}$$

where w_i^l and w_i^g are the local and the global model of the client i, $\mathcal{L}(w_i^l; x_i, y_i)$ is the performance of the local model and $\mathcal{L}(w^g; x_i, y_i)$ is the performance of the global model on clients data.

This approach might produce several overfitted models, which is not the desired behavior expected by the FL environments. Thus, to enhance the system performance while taking advantage of both approaches, ACSP-FL divides the model of each client into two parts. One set of layers will be part of the collaboratively trained global model (w^g). The other layers will compose the personalized model (w_i^l) to be trained only locally on the client's data. Therefore, the model of the client $i \in C$ is a composition as $w_i = [w^g, w_i^l]$.

To define which layers will be trained cooperatively, ACSP-FL uses a function $\mathcal{K}(w, L)$ to cut the model according to the layers $L = \{l_1, l_2, \ldots, l_m\} \in w$, where each layer $l_j \in L$ represents a layer of the global model. ACSP-FLcan define the number of layers statically, in which the same number is used for all clients in all communication rounds or dynamically based some algorithm or function considering the performance of a client in a particular round.

Fig. 3 shows how ACSP-FL builds the model in each client combining the shared layers (i.e., the global piece w^g) with the local layers personalized in each client (i.e., the local piece w_i^l). In this example, $L = \{l_1, l_2\}$ defines that the first two layers, the blue neurons, are the shared global model $w^g = \mathcal{K}(w, L)$, while the remaining part, the red neurons, will be personalized on its local data. Therefore, combining these pieces together each client builds its own model $w_i = [w^g, w_i^l]$.

Eventually, layer sharing among clients enables better performance and reduces communication overhead in both uplink and downlink with fewer parameters transmission over the network.

3.5. Proposed algorithms

This section describes the procedures employed by the server and the clients with ACSP-FL. This way, Algorithm 1 shows how the server selects the clients for each round and how it defines the pieces of the model that will be shared, while Algorithm 2 presents how the clients personalize the model and fit it on its local data and also how they evaluate the trained model and report its performance for the server.

In particular, the server initiates the procedure defining the number of clients participating in the federated learning procedure C and the number of communication rounds T. Then, it initializes the model w and selects all clients for the first communication rounds (Lines 2–3). Thus, for each communication and client selected (i.e., $i \in S$) the server defines the shared layers among all clients (Line 6), cuts the model based on the set of layers defined (Line 7) and sends it to each client to perform the local fit (Line 8).

Hereafter, the server receives the trained pieces w_i^g of each client and computes the total size of the dataset D based on the size reported by each client d_i to perform the aggregation (Lines 9–10). Moreover, with the aggregated model, the server sends it to be evaluated by each client and to receive their performance metric (Line 11). Then, based on the performance of each client, the server applies the adaptive client

Algorithm 1: Server Function

```
   input : C /* Set of clients it the system                    */
           T /* Number of communication rounds                  */
1  Server procedure (C, T):
       // initialize the model
2      w ← RandomInit();
       // select all clients for the first round
3      S ← C;
4      for t ∈ {1, 2, 3, ... T} do
5          foreach i ∈ S do
               // defines the part of the model that will be
                  shared
6              L ← SharedLayers(w);
               // cut the model based on the layers L
7              w^g ← K(w, L);
               // Send the shared part to clients i ∈ S
8              w_i, |d_i| ← LocalTrain(i, w^g);
9          D ← Σ_{i∈S} d_i // aggregates the received models
10         w ← Σ_{i∈S} (|d_i| / D) w_i;
           // send the aggregated model for evaluation
11         A ← Evaluate(w, C);
           // compute the set of selected clients for the
              next round
12         S ← ClientSelection(C, A, t);
13 ClientSelection (C, A, t):
14     S' ← [];
15     for i ∈ C do
           // filter clients based on condition π(·)
16         S' ⋃ π(i, A);
       // filter clients based on the decay factor
17     S ← ⋃_{j=0}^{φ(S',t)} S'[j];
       // return selected clients
18     return S
```

Algorithm 2: Client training

```
   data: i /* client identification                             */
         d_i /* local dataset of client i                       */
         w_i^l /* local model of client i                       */
1  LocalTrain (i, w^g):
       // combines the local model with the local one
2      w_i ← [w^g, w_i^l];
3      for epoch ∈ {1, 2, 3, ..., τ} do
           // split the train data in batches
4          foreach (x_i, y_i) ∈ d_i do
               // compute the loss for each batch
5              L(w_i; x_i, y_i);
       // return trained piece and the size of d_i
6      return w_i^g, |d_i|
7  Evaluate (w^g, i):
       // combines the local model with the local one
8      w_i ← [w^g, w_i^l];
       // evaluate the model on test data
9      A_i ← Eval(w_i, N_i);
       // return the performance for the server
10     return A_i
```

selection procedure to define the set of clients S for the next communication round (Line 12). The entire procedure applied by ACSP-FL for the client selection is described from Line 14 to Line 18.

On the other hand, when each selected client $i \in S$ receives the local train request, it produces the model w to be trained by combining the shared piece (i.e., global model w^g) with the local piece (i.e., w_i^l) described in Line 2. Then, for each local epoch, the client splits the local train dataset into batches (x_i, y_i) to fit the model (Lines 4–5). With the model trained, the client reports the shared piece w_i^g to the server with the size of its train dataset d_i. These are the steps to train the model considering the personalization approach that combines the local model with the global one. It is essential to highlight that in such communication procedure (i.e., it sends the pieces of the model for the client and reports it back to the server) ACSP-FL reduces the communication overhead because fewer parameters are shared, saving bandwidth in both directions uplink and downlink, while keeping the system accuracy high.

Lastly, when clients receive the evaluation request from the server to perform the distributed evaluation of the model, each client also needs to build the model based on the local and shared pieces (Line 8) to evaluate it on its test data (Line 9). This procedure returns the performance metric $\mathcal{A}_i$ used to evaluate the model's performance in each client. Then, this metric backs to the server (Line 10), which will use it to perform the adaptive client selection method.

4. Performance evaluation

This section presents the evaluation of ACSP-FL against the state-of-the-art works FedAvg [10], POC [7], and DEEV [9], analyzing the model's performance, client selection, as well as communication and processing overhead. For this purpose, we used a Docker Swarm to create multiple virtual machines (VMs) organized as either VM managers or VM workers. Additionally, each application, namely, server and client, is deployed on the VMs as containers. Each type of VM is responsible for running a specific type of application. Thus, the VM managers run the server application, while the VM workers run one or more client applications (i.e., containers).

4.1. Evaluation environment

A container-based environment (i.e., dockerized) runs the evaluation, where the main idea is to separate the server and client applications onto different virtual machines. To achieve this, we used a Docker Swarm to create multiple virtual machines (VMs), acting as VM managers or VM workers. Furthermore, each application, namely, server and client, is deployed on the VMs as containers. Each type of VM is responsible for executing a specific type of application. Therefore, the VM managers run the server application, while the VM workers run one or more client applications (i.e., containers).

Client containers are responsible for storing local data, training the model locally with their data, compressing the model to enhance communication efficiency, sharing the trained model with the server for collaborative learning, and updating the model after the server performs aggregation. Communication between the clients and the server is facilitated through gRPC using the Flower framework [30]. On the other hand, the server application aggregates the models received from the clients, evaluates the overall model performance, selects a set of clients for training in the next round of communication, and ultimately provides communication logs, resource usage, and performance metrics. These logs are obtained using the Docker engine API.

A *docker-compose* file sets up the environment where different parameters can be passed through each container using environment variables, including neural network configuration, aggregation approach, compression method, and resource allocation (i.e., CPU and RAM limits). It is worth noting that the network attribute in the *docker-compose* file can define the usage of different communication technologies. Additionally, the customization of CPU and RAM limitations for each container can determine heterogeneous clients.

4.2. Dataset, application, and model

To evaluate the performance of the FL algorithms, we consider a Human Activity Recognition (HAR) application using sensor data

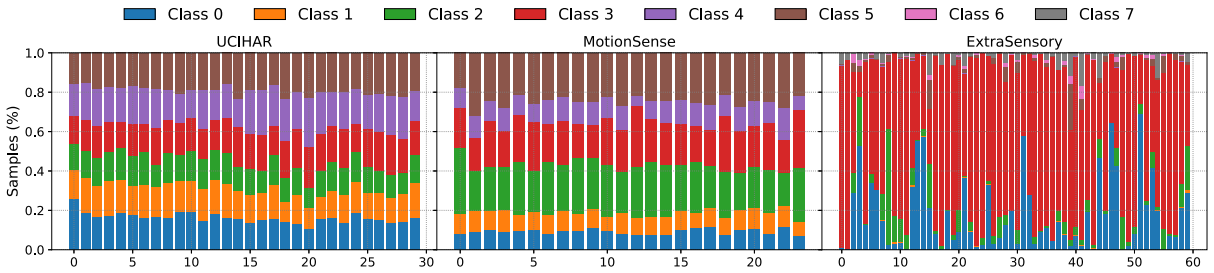


Fig. 4. Visualization of classes distribution by client for each dataset.

Table 2
Dataset characteristics.

	Dataset		
	UCI-HAR	MotionSense	ExtraSensory
# Clients	30	24	60
# Classes	6	6	8
# Features	561	7	277
Samples per client (avg.)	274	47 095	5015
Client with min. # samples	224	40 804	1280
Client with max. # samples	327	57 559	9596

collected by smartphones. In this evaluation, we considered three HAR datasets:

- **UCI-HAR Dataset**: is a widely recognized benchmark for human activity recognition, featuring accelerometer and gyroscope data from smartphones worn by approximately 30 users. These users perform activities including walking upstairs, downstairs, sitting, standing, and lying. The dataset provides time-series sensor data and corresponding activity labels, serving as a fundamental resource for assessing and developing human activity recognition algorithms in the machine learning and data mining communities [31].
- **MotionSense Dataset**: is a human activity recognition containing sensor data from accelerometers and gyroscopes of smartphones worn by 24 users engaged in activities such as walking, jogging, sitting, standing, and climbing stairs. This dataset provides time-series sensor data across three dimensions (x, y, z) and corresponding activity labels, making it a robust tool for activity recognition and sensor data analysis research [32].
- **ExtraSensory Dataset**: is a unique dataset designed for mobile and wearable sensing research, comprising sensor data collected from smartphones and wearable devices used by 60 users. It encompasses data from various sensors, including accelerometers, GPS, light sensors, and more, along with rich contextual information such as location, time, and weather conditions. Participants engage in diverse activities, making this dataset invaluable for context-aware applications, activity recognition research, and exploring relationships between sensor data and user context [33].

Fig. 4 displays the class distribution for each client across various datasets. As observed, the UCI-HAR and MotionSense datasets exhibit a comparable number of clients, as well as a similar distribution, suggesting identically distributed (IID) characteristics. In contrast, the ExtraSensory dataset features a larger number of clients, and the class distribution among these clients is non-identically distributed (Non-IID). Table 2 describes more details for each dataset, including number of clients, average samples per client, number of features and also minimum and maximum number of samples in clients.

A suitable model for the HAR problem is the *MultiLayer Perceptron* (MLP) [29] composed of three hidden layers, with 256 units per hidden layer [34]. The following parameters adjust the MLP: *Stochastic Gradient Descent* (SGD) as a method to update the weights of the MLP model and *Sparse Categorical Crossentropy* as a loss function. We want to optimize the accuracy score as it is a classification task. The objective is to train the global model (MLP) on local customer data, evaluate it in a distributed way, and achieve good performance.

4.3. Assessed metrics

To evaluate the performance and also the communication and processing overhead of the solutions, the following metrics were defined:

- **Distributed accuracy**: measures the convergence of the model. Thus, the accuracy is distributively computed in each communication round. Each client evaluates the aggregated model in its local test dataset and reports the result to the central server, which computes the average accuracy considering all clients' reports. The fewer rounds needed to reach the higher accuracy, the faster the solution will converge.
- **Overhead**: measures the time spent in each round for each solution, considering: *(i)* the time to receive the model from the server, *(ii)* the processing time to train the model, and *(iii)* time to transmit the trained model back to the server. To compute this metric, we define the FedAvg's latency as the baseline. Thus, the results represent the percentage of reduction compared to the latency of FedAvg.
- **Data transmitted**: this metric evaluates the data transmitted, specifically the TX bytes, taking into account both the overall data transmitted and the data transmitted per client. To calculate this metric, we utilized the size of the model sent by each client in bytes during each round.
- **Convergence time**: shows the total time, in seconds, taken by each solution to complete 100 communication rounds. In essence, it quantifies the time required for each solution to conclude its execution.
- **Efficiency**: this metric evaluates the efficiency of the solution to reach the provided accuracy by considering a weighted sum of the average accuracy and the overhead reduction. In particular, the weighted sum is calculated as $\alpha \times \mathcal{A}_{mean} + \beta \times overhead\ reduction$. These parameters can be adjusted according to the importance of the accuracy and the communication, but in this evaluation we set the value of both parameters to 0.5.
- **Accuracy distribution**: shows the final test accuracy in each client in the last communication round. Thus, we can show whether all clients had good performance or some clients with high performance and others with poor ones. In other words, with this metric, we want to see how homogeneous the performance of all clients is.
- **Client selection**: this metric assesses how often the clients were selected for the adaptive client selection solutions. The higher number of clients means higher communication overhead. The key idea is to understand the average number of times clients participated in the training.

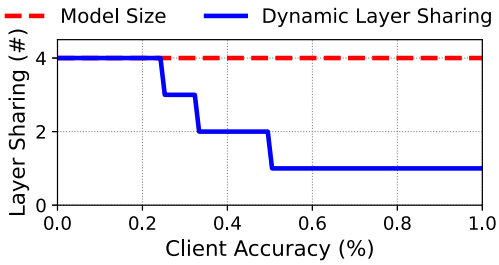


Fig. 5. Example of dynamic layer definition function based on client's accuracy.

4.4. ACSP-FL analysis

In this section, we present an exploratory analysis of the methods implemented by ACSP-FL to assess how personalization, partial model sharing, and the client selection mechanism impact its performance in terms of *(i)* accuracy; *(ii)* communication overhead; *(iii)* convergence time; and *(iv)* efficiency. Fig. 6 illustrates the distributed accuracy for each evaluated dataset. For clarity, we have named the solutions based on the number of layers shared between clients and the server. It is essential to note that ACSP-FL FT represents the fine-tuned approach, where each client decides whether to retain its local model or adopt the global one, with no partial model sharing, while ACSP-FL PMS (#) represents the number of layer of partial model sharing, ACSP-FL DLD represents the dynamic layer definition function and ACSP-FL ND represents the approach without using personalization and the decaying function.

The dynamic layer definition employed by ACSP-FL ND is given by:

$$PMS = \begin{cases} 4 & \mathcal{A}^t \leq 0.25 \\ \left\lceil \frac{1}{\mathcal{A}^t} \right\rceil & otherwise \end{cases} \tag{9}$$

where 4 is the total number of layers of the model used in the evaluation and $\mathcal{A}^t$ is the accuracy of the client at the communication round t. Fig. 5 shows an example of how to determine the number of layers to share based on the clients' accuracy, as the accuracy increases fewer layers are shared giving higher importance to the local model but also considering the knowledge of the last layer trained cooperatively.

The results indicate that, despite the number of layers shared, ACSP-FL can still converge for each dataset (refer to Fig. 6). However, the ACSP-FL ND variant cannot achieve the same accuracy as the other variations due to the absence of model personalization. On the other hand, Fig. 7 illustrates the data transmitted by each solution, revealing three main aspects. Firstly, the ACSP-FL variant that employs the decaying function reduces the data transferred as the training phase progresses, as fewer clients are selected when the model converges. In contrast, ACSP-FL ND, which does not use the decaying function, experiences constant communication overhead across all datasets. The second aspect is that, with fewer layers shared by ACSP-FL PMS variants, less data is transferred. Lastly, ACSP-FL DLD, which determines the number of layers shared in each round by each client according to its accuracy, adapts itself to reduce communication overhead. Therefore, it proves to be a suitable solution, as no hyperparameter exploration is needed for different scenarios.

The performance of ACSP-FL variants for other metrics is summarized in Table 3, which presents the performance of each variant considering accuracy, total data transmitted, average data transmitted per client, convergence time, and the efficiency metric. The results indicate that personalization enhances model accuracy, while partial model sharing reduces overhead by up to 99%. Additionally, as less data is transmitted, it also reduces the convergence time for variants that utilize partial model sharing. It is important to note that ACSP-FL DLD slightly increases the convergence time due to the dynamic layer definition function that needs to be computed every single round. Finally, the efficiency metric shows that the combination of personalization and partial model sharing improves the efficiency of the solution. The closer the metric is to 1, the better the efficiency of the solution, indicating its ability to deliver a reduction in communication overhead similar to its accuracy.

In summary, by combining these three approaches *(i)* adaptive client selection with the decaying function; *(ii)* model personalization; and *(iii)* partial model sharing, ACSP-FL reaches higher accuracy level in early stages of the federated cycle. Empowered by the personalization method, fewer clients start to be selected due to the adaptive client selection, and in this whole cycle, less data is transmitted over the network due to the partial model sharing, demonstrating how efficient ACSP-FL is.

4.5. ACSP-FL vs. literature analysis

This evaluation aims to assess the federated learning efficiency of each solution: FedAvg [10], POC [7], DEEV [9], OORT [22], and ACSP-FL with personalization and partial model sharing using the dynamic layer definition (i.e., ACSP-FL DLD).

Firstly, Fig. 8 shows distributed accuracy evaluation for the three datasets analyzed for each solution. An exploratory analysis was performed to define the best hyper-parameters for the literature solutions. Therefore, for the OORT and POC solution, which need to define the k number of clients to be selected per round, for $k = 50\%$, we had the best cost-benefit between performance and communication overhead. On the other hand, we define the FedAvg as a baseline and thus define that 100% of clients will always be selected in each round. Furthermore, for ACSP-FL, we set the value for $decay = 0.005$.

As seen in Fig. 8, ACSP-FL improved the model's performance for all datasets compared to the other solutions. This results from the model personalization used by ACSP-FL that considers both local and global models to decide which is more suitable for each client. In

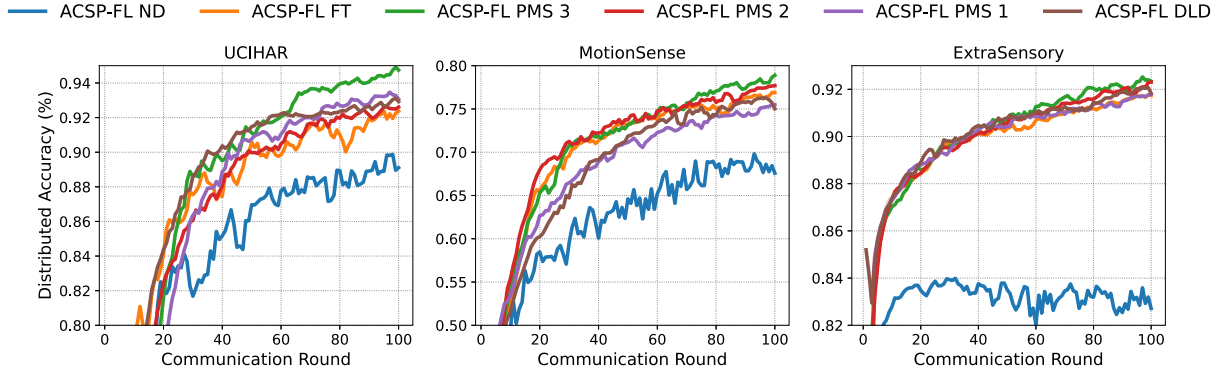


Fig. 6. Test accuracy in function of the communication round for each dataset.

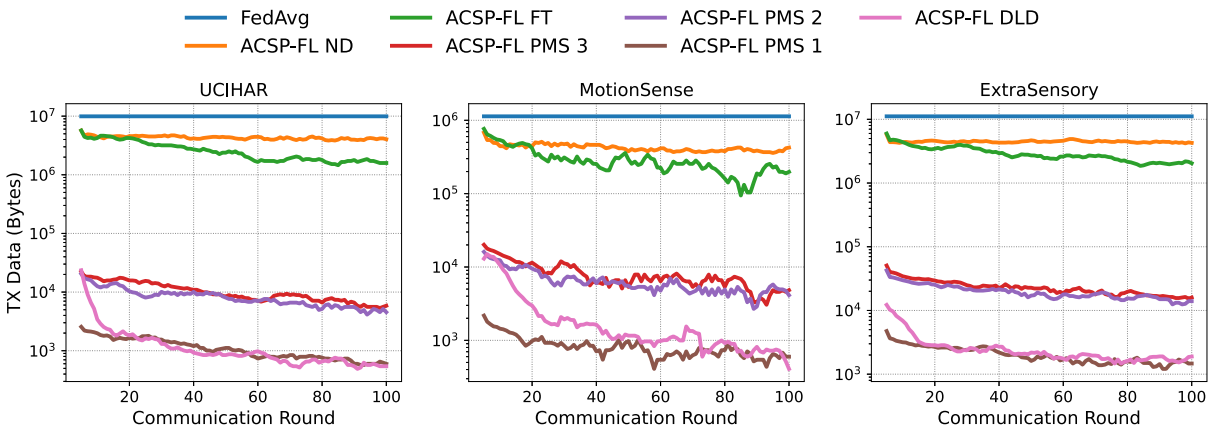


Fig. 7. Analysis of communication overhead considering the partial model sharing and also the decaying function.

Table 3
Summary of assessed metrics for ACSP-FL variations.

		Metrics				
	Solutions	Accuracy (%)	TX bytes (Mb)	TX bytes per client (Mb)	Convergence time (s)	Efficiency
UCI-HAR	ACSP-FL ND	0.90	434	14.4	19.14	0.71
	ACSP-FL FT	0.89	268	8.9	18.15	0.79
	ACSP-FL PMS 3	0.92	1.07	0.03	8.34	0.96
	ACSP-FL PMS 2	0.90	0.85	0.02	8.31	0.95
	ACSP-FL PMS 1	0.91	0.11	0.003	8.30	0.95
	ACSP-FL DLD	0.92	0.22	0.007	9.60	0.95
MotionSense	ACSP-FL ND	0.70	43.1	1.79	221.27	0.67
	ACSP-FL FT	0.71	30.0	1.25	221.27	0.74
	ACSP-FL PMS 3	0.71	0.82	0.03	209.63	0.88
	ACSP-FL PMS 2	0.71	0.66	0.02	208.13	0.87
	ACSP-FL PMS 1	0.68	0.08	0.003	210.85	0.87
	ACSP-FL DLD	0.70	0.24	0.010	216.76	0.87
ExtraSensory	ACSP-FL ND	0.92	452	7.53	40.72	0.74
	ACSP-FL FT	0.92	298	4.97	38.39	0.82
	ACSP-FL PMS 3	0.95	2.38	0.039	31.48	0.95
	ACSP-FL PMS 2	0.92	2.03	0.033	30.99	0.95
	ACSP-FL PMS 1	0.92	0.21	0.003	30.77	0.95
	ACSP-FL DLD	0.93	0.28	0.004	38.91	0.95

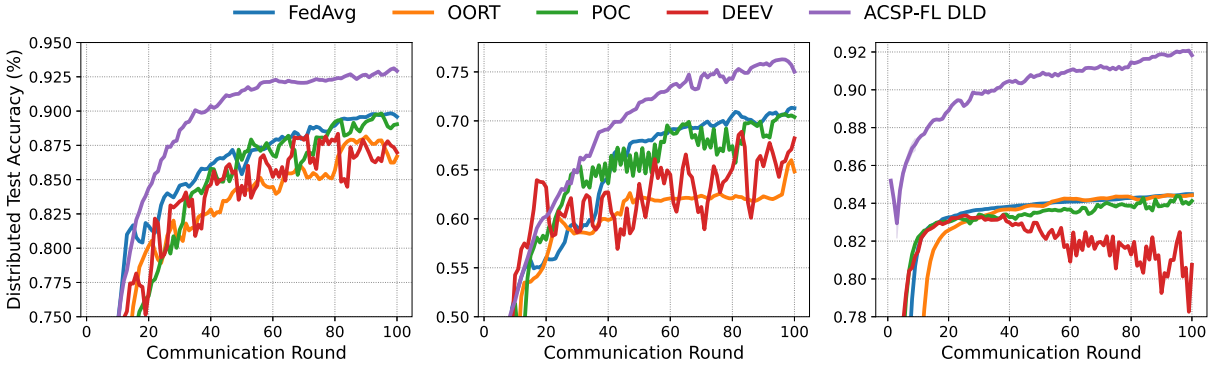


Fig. 8. Test accuracy in function of the communication round for each dataset.

addition, we can also see the advantages of layer sharing among clients to improve ACSP-FL performance. In particular, ACSP-FL with both approaches increases the model's accuracy in up to 8% when compared to FedAvg, POC, OORT, and DEEV. It is worth noticing that ACSP-FL does not overfit the model for each client since when combining the personalization method with the layer sharing, a collaborative training is introduced, consequently ACSP-FL improving its accuracy.

Another important metric to be assessed is the overhead produced by each solution, as depicted in Fig. 9. The benefits of the dynamic client selection and layer sharing used by ACSP-FL are evident, resulting in a reduction of approximately 90% compared to FedAvg's overhead. In this study, overhead represents the time spent for each solution to receive a model from the server, perform a training round, and report it back to the server. Therefore, as fewer clients are selected according to the decay effect implemented by ACSP-FL, the server must wait for fewer clients to proceed to the next communication round. Furthermore, as ACSP-FL also shares fewer layers than the other solutions, it further reduces the communication overhead produced by model sharing, consequently lowering the overhead.

Comparing ACSP-FL with POC and OORT solutions that require determining the number of clients participating in each communication round in advance we observe that the overhead reduction is

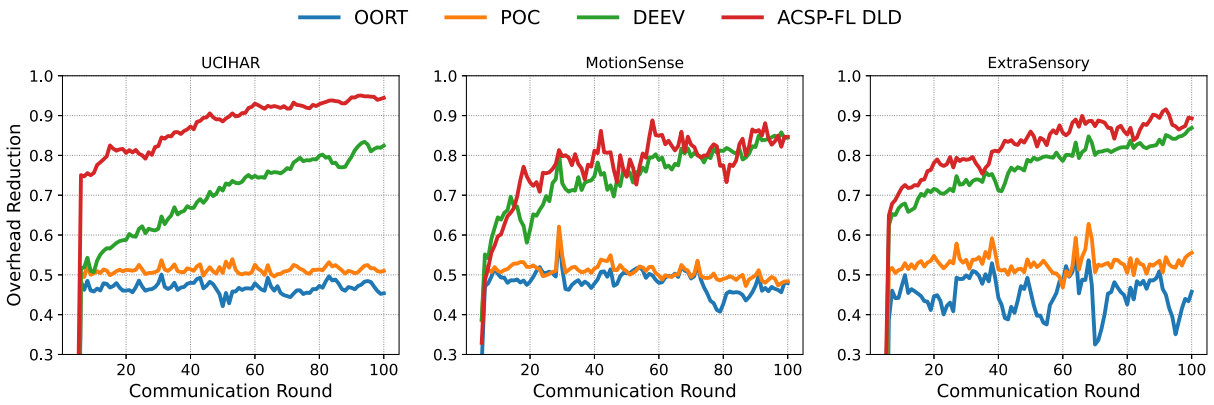


Fig. 9. Latency produced by each solution for each dataset.

Table 4
Summary of assessed metrics for ACSP-FL and literature solutions.

		Metrics				
	Solutions	Accuracy (%)	TX bytes (Mb)	TX bytes per client (Mb)	Convergence time (s)	Efficiency
UCI-HAR	FedAvg	0.89	992.52	33.08	19.17	0.42
	OORT	0.87	501.22	16.70	20.36	0.70
	POC	0.88	501.22	16.70	18.70	0.69
	DEEV	0.87	314.29	10.47	18.39	0.69
	ACSP-FL DLD	0.92	0.22	0.007	9.66	0.95
MotionSense	FedAvg	0.70	113.26	4.71	240.82	0.39
	OORT	0.65	57.19	2.38	249.13	0.62
	POC	0.70	57.19	2.38	236.58	0.61
	DEEV	0.67	32.98	1.37	212.78	0.73
	ACSP-FL DLD	0.75	0.24	0.010	216.76	0.87
ExtraSensory	FedAvg	0.84	1114.17	18.56	42.91	0.44
	OORT	0.84	562.65	9.37	47.31	0.69
	POC	0.84	562.65	9.37	40.38	0.68
	DEEV	0.81	311.41	5.19	35.73	0.79
	ACSP-FL DLD	0.92	0.28	0.004	38.91	0.95

proportional to the number of clients selected by POC. This reduction remains almost constant since it does not change that number over the communication rounds. However, OORT achieves a lower reduction, as it needs to compute the statistical metric for each client in every single communication round. Lastly, when comparing ACSP-FL with DEEV, a slight improvement results from layer sharing. Consequently, less time is spent transferring the model over the network. In particular, ACSP-FL reduces latency by approximately 30% and 10% compared to POC and DEEV.

Table 4 provides a comprehensive summary of the assessed metrics for ACSP-FL compared to other solutions in the literature, addressing datasets such as UCI-HAR, MotionSense, and ExtraSensory. In terms of accuracy, ACSP-FL DLD stands out, achieving 92% in the UCI-HAR dataset, 75% in MotionSense, and 92% in ExtraSensory, surpassing other approaches. Additionally, when analyzing communication efficiency, ACSP-FL DLD achieves a notable reduction in transmitted data and convergence time. Compared to FedAvg, it exhibits a reduction of approximately 99% in transmitted bytes in UCI-HAR, 99% in MotionSense, and 97% in ExtraSensory. Furthermore, the ACSP-FL DLD solution excels in efficiency, with a significant reduction in communication overhead, making it a promising option for performance optimization in federated learning environments.

Finally, the results show that ACSP-FL DLD excels in striking a balance between communication overhead reduction and model accuracy. The significant efficiency gains, as indicated by metrics exceeding 0.9 in all cases, underscore the efficacy of the dynamic layer definition mechanism employed by ACSP-FL DLD. By adapting the number of shared layers based on client accuracy, ACSP-FL DLD optimizes communication efficiency, showcasing its robust performance in federated learning scenarios.

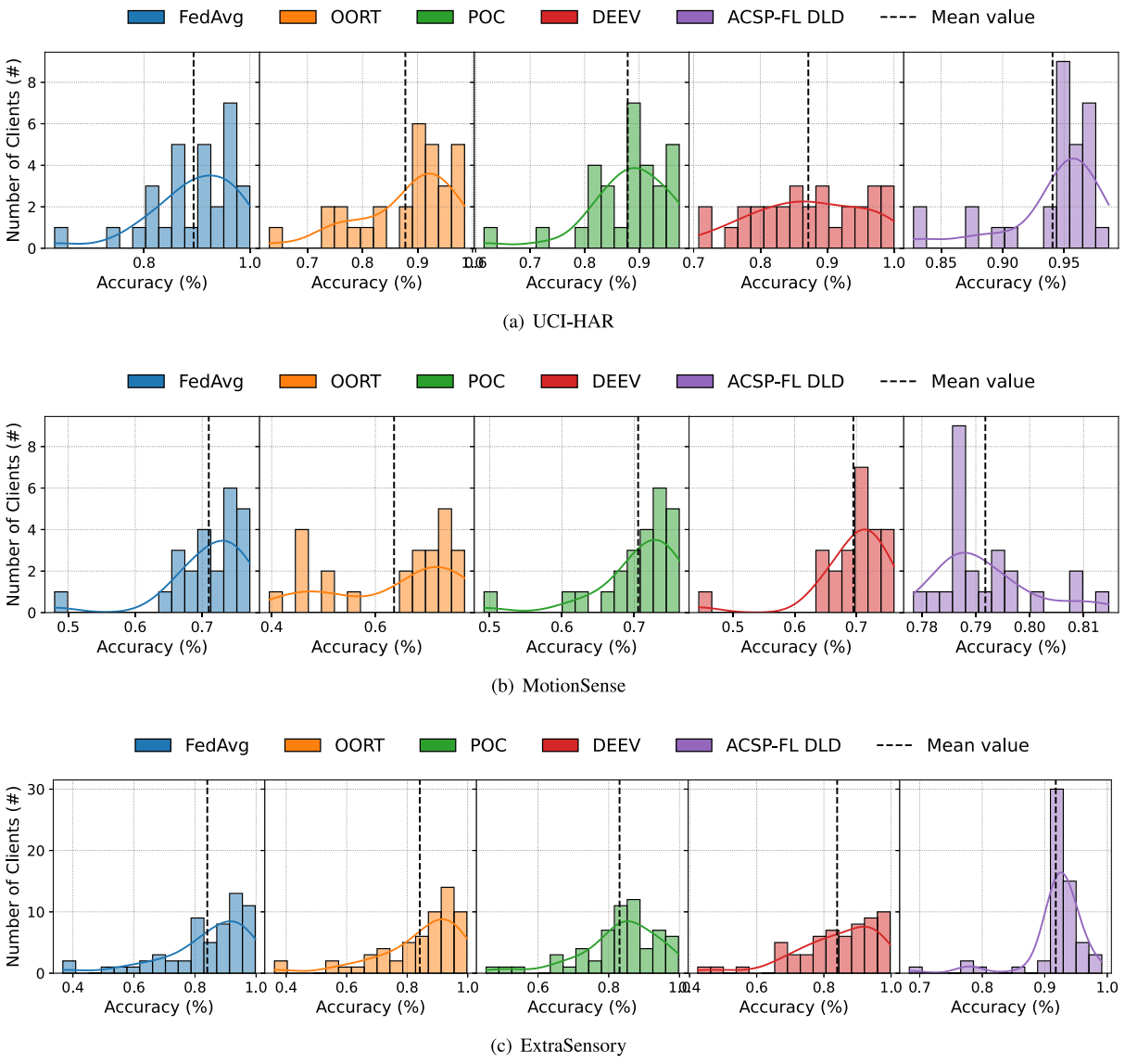
4.6. Clients performance analysis

Fig. 10 illustrates the performance of each client for each dataset at the final communication round. The implemented personalization methods in ACSP-FL are evident in the higher accuracy achieved for the majority of clients. This metric is crucial for depicting the overall model performance, as relying solely on the average accuracy of clients may hide the performance of specific clients with notably low accuracy.

Therefore, when considering the number of clients higher than the average accuracy, we can see that for ACSP-FL, more clients achieve accuracy higher than the average one, and the average value is higher than the average value of literature solutions. Moreover, the lowest accuracy for the clients is also higher in ACSP-FL. For instance, when considering Fig. 10(c), the client with lower accuracy in FedAvg is approximately 40%. However, when comparing to ACSP-FL, the client with the lowest performance achieves accuracy higher than 70%. This data shows how efficient ACSP-FL is in empowering clients to achieve better results with fine-tuning and layer sharing.

4.7. Client selection analysis

Fig. 11 presents the client selection frequency for each solution. As discussed previously, FedAvg exhibits a 100% selection rate for all clients across 100 rounds, a consequence of the simulation parameters that involve the participation of all clients in each communication round. In contrast, POC and OORT solutions specify a fixed number of clients for participation in each round, differing in their selection mechanisms. The results reveal that OORT selects approximately 40% more clients compared to POC. However, for the other 50% of clients, POC increases their participation frequency in the training phase. This



(a) UCI-HAR

(b) MotionSense

(c) ExtraSensory

Fig. 10. Accuracy distribution for each client at the last communication round.

outcome stems from POC's selection mechanism, exclusively utilizing the poorest-performing clients, leading to a consistent set of clients participating in subsequent rounds after the initial 50 rounds.

When examining ACSP-FL with adaptive client selection, personalization, and layer sharing, clients are chosen less frequently for model training. This is a consequence of improved client performance, as depicted in Fig. 10 where a greater number of clients exhibit performance surpassing the average. Consequently, fewer clients remain eligible for selection and model training, given that these solutions exclusively consider clients with performance lower than the average model performance. In comparison to DEEV, ACSP-FL achieves a reduction of up to 30% in the frequency of client selection. However, when juxtaposed with OORT, POC, and FedAvg, this reduction can reach up to 50%, 60%, and 70%, respectively.

Upon examining the maximum number of selections, it is evident that within ACSP-FL, clients were selected up to 50 times for UCI-HAR, approximately 35 times for MotionSense, and around 50 times for ExtraSensory. In contrast, other solutions reached up to 80 selections, as seen in the case of POC for ExtraSensory and UCI-HAR. It is noteworthy that DEEV achieves a similar performance when compared to ACSP-FL for some datasets. It is consequence of its adaptive client selection mechanism. However, the slight improvement in ACSP-FL is a result of its personalization method that enhances its performance.

Finally, selecting fewer clients and fewer times improves the overall system performance because it will also reduce the communication and processing overheads. In addition, selecting clients fewer times consumes fewer resources for each client since the server does not frequently select them to train the model. Thus, when considering resource constraints scenarios, ACSP-FL seems an excellent choice to preserve the clients' resources.

With these results, we can conclude that: *(i)* ACSP-FL provides an efficient adaptive client selection mechanism that reduces the overall latency of the system; *(ii)* the personalization employed by name enables clients to achieve better performance; *(iii)* personalization with layer sharing helps to improve model performance while reducing the communication overhead since fewer layers are transmitted over the network.

5. Conclusion

This work presented ACSP-FL, a federated learning solution with personalization and adaptive client selection based on the performance

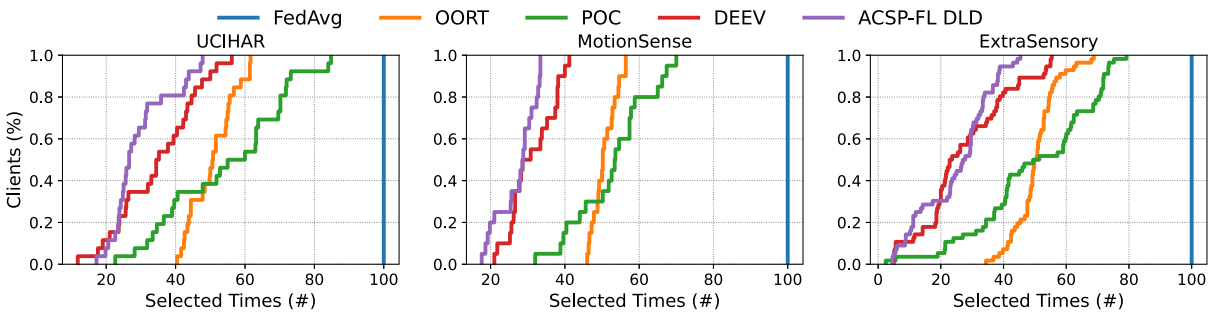


Fig. 11. Evaluation of how many times clients were selected by each solution.

of the distributed model. ACSP-FL also implements a gradual decay function to reduce the number of clients selected for each round according to model performance. This design leads to faster convergence and reduces processing and communication *overhead*. Also, it implements a personalization method that enables clients to fine-tune the model for their dataset, and by using a layer sharing mechanism ACSP-FL transmits fewer data (i.e., parameters of the model) over the network, consequently reducing the communication overhead even further.

To evaluate ACSP-FL's performance, we used three different datasets for human activity recognition created with data from *smartphones* sensors. The results showed that ACSP-FL reduces the communication overhead by up to 95% and also the latency to train the federated model by more than 90% concerning literature solutions. Furthermore, ACSP-FL also increases the model's performance by at least 10% with its personalization and layer-sharing approach compared to other literature solutions.

Privacy is one key aspects in FL services, which not only aligns with ethical considerations but also fosters trust among participants, encouraging widespread participation in collaborative machine learning endeavors while preserving the confidentiality of individual data. In this way, it is important to stress that additional methods to improve even further clients' privacy can be implemented in ACSP-FL such as secure aggregation and differential privacy based algorithms.

Finally, as future work, information about device capabilities will be incorporated to enable context-aware selection of battery, CPU and memory usage, and even network quality of devices. Model compression algorithms will also be developed to reduce the solution's communication overhead further.

CRediT authorship contribution statement

Allan M. de Souza: Writing – review & editing, Writing – original draft, Visualization, Validation, Methodology, Investigation, Formal analysis, Data curation, Conceptualization. **Filipe Maciel:** Writing – review & editing. **Joahannes B.D. da Costa:** Writing – review & editing. **Luiz F. Bittencourt:** Writing – review & editing. **Eduardo Cerqueira:** Writing – review & editing. **Antonio A.F. Loureiro:** Writing – review & editing. **Leandro A. Villas:** Writing – review & editing.

Declaration of competing interest

The authors declare the following financial interests/personal relationships which may be considered as potential competing interests: Leandro A. Villas reports financial support was provided by PPI-Softex. If there are other authors, they declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

Data and code are available in the repository linked to the paper.

Acknowledgments

This project was supported by the Ministry of Science, Technology, and Innovation of Brazil, with resources granted by the Federal Law 8.248 of October 23, 1991, under the PPI-Softex. The project was coordinated by Softex and published as Intelligent agents for mobile platforms based on Cognitive Architecture technology [01245.013778/2020-21].

References

[1] P. Porambage, J. Okwuibe, M. Liyanage, M. Ylianttila, T. Taleb, Survey on multi-access edge computing for Internet of Things realization, IEEE Commun. Surv. Tutor. 20 (4) (2018) 2961–2991.

[2] N. Abbas, Y. Zhang, A. Taherkordi, T. Skeie, Mobile edge computing: A survey, IEEE Internet Things J. 5 (1) (2017) 450–465.

[3] W.Y.B. Lim, N.C. Luong, D.T. Hoang, Y. Jiao, Y.-C. Liang, Q. Yang, D. Niyato, C. Miao, Federated learning in mobile edge networks: A comprehensive survey, IEEE Commun. Surv. Tutor. 22 (3) (2020) 2031–2063.

[4] S. AbdulRahman, H. Tout, H. Ould-Slimane, A. Mourad, C. Talhi, M. Guizani, A survey on federated learning: The journey from centralized to distributed on-site learning and beyond, IEEE Internet Things J. 8 (7) (2020) 5476–5497.

[5] X. Zhang, X. Zhu, J. Wang, H. Yan, H. Chen, W. Bao, Federated learning with adaptive communication compression under dynamic bandwidth and unreliable networks, Inform. Sci. 540 (2020) 242–262.

[6] O.A. Wahab, A. Mourad, H. Otrok, T. Taleb, Federated machine learning: Survey, multi-level classification, desirable criteria and future directions in communication and networking systems, IEEE Commun. Surv. Tutor. 23 (2) (2021) 1342–1397.

[7] Y.J. Cho, J. Wang, G. Joshi, Client selection in federated learning: Convergence analysis and power-of-choice selection strategies, 2020, arXiv preprint arXiv:2010.01243, URL: https://arxiv.org/pdf/2010.01243.pdf.

[8] A.Z. Tan, H. Yu, L. Cui, Q. Yang, Towards personalized federated learning, IEEE Trans. Neural Netw. Learn. Syst. (2022) 1–17, URL: https://ieeexplore.ieee.org/document/9743558.

[9] A.M. de Souza, L. Bittencourt, E. Cerqueira, A. Loureiro, L. Villas, Dispositivos, Eu Escolho Vocês: Seleção de Clientes Adaptativa para Comunicação Eficiente em Aprendizado Federado, in: Anais do XLI Simpósio Brasileiro de Redes de Computadores e Sistemas Distribuídos, SBC, Porto Alegre, RS, Brasil, 2023, pp. 1–14, URL: https://sol.sbc.org.br/index.php/sbrc/article/view/24525.

[10] B. McMahan, E. Moore, D. Ramage, S. Hampson, B.A. y Arcas, Communication-efficient learning of deep networks from decentralized data, in: Artificial Intelligence and Statistics, PMLR, 2017, pp. 1273–1282.

[11] S.M. Shah, V.K.N. Lau, Model compression for communication efficient federated learning, IEEE Trans. Neural Netw. Learn. Syst. 34 (9) (2023) 5937–5951.

[12] Y. Lin, S. Han, H. Mao, Y. Wang, B. Dally, Deep gradient compression: Reducing the communication bandwidth for distributed training, in: International Conference on Learning Representations, 2018, pp. 1–14, URL: https://openreview.net/pdf?id=SkhQHMW0W.

[13] M.M. Amiri, D. Gunduz, S.R. Kulkarni, H.V. Poor, Federated learning with quantized global model updates, 2020, arXiv preprint arXiv:2006.10672.

[14] J. Bernstein, Y.-X. Wang, K. Azizzadenesheli, A. Anandkumar, signSGD: Compressed optimisation for non-convex problems, in: International Conference on Machine Learning, PMLR, 2018, pp. 560–569.

[15] F. Sattler, S. Wiedemann, K.-R. Müller, W. Samek, Robust and communication-efficient federated learning from non-I.I.D. data, IEEE Trans. Neural Netw. Learn. Syst. 31 (9) (2020) 3400–3413.

[16] N. Ström, Scalable distributed DNN training using commodity GPU cloud computing, in: Interspeech 2015, 2015, pp. 1–5, URL: https://www.amazon.science/publications/scalable-distributed-dnn-training-using-commodity-gpu-cloud-computing.

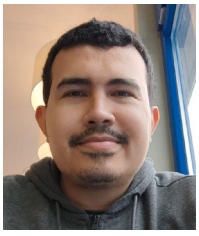
[17] O. Shahid, S. Pouriyeh, R.M. Parizi, Q.Z. Sheng, G. Srivastava, L. Zhao, Communication efficiency in federated learning: Achievements and challenges, 2021, http://dx.doi.org/10.48550/arXiv.2107.10996, arXiv preprint arXiv:2107.10996.
[18] G.D. Németh, M.A. Lozano, N. Quadrianto, N.M. Oliver, A snapshot of the frontiers of client selection in federated learning, Trans. Mach. Learn. Res. (2022) URL: https://openreview.net/forum?id=vwOKBldzFu.
[19] T. Nishio, R. Yonetani, Client selection for federated learning with heterogeneous resources in mobile edge, in: ICC 2019 - 2019 IEEE International Conference on Communications, ICC, 2019, pp. 1–7.
[20] J. Goetz, K. Malik, D. Bui, S. Moon, H. Liu, A. Kumar, Active federated learning, 2019, arXiv preprint arXiv:1909.12641.
[21] J. Kang, Z. Xiong, D. Niyato, S. Xie, J. Zhang, Incentive mechanism for reliable federated learning: A joint optimization approach to combining reputation and contract theory, IEEE Internet Things J. 6 (6) (2019) 10700–10714.
[22] F. Lai, X. Zhu, H.V. Madhyastha, M. Chowdhury, Oort: Efficient federated learning via guided participant selection, in: 15th USENIX Symposium on Operating Systems Design and Implementation, OSDI 21, USENIX Association, 2021, pp. 19–35, URL: https://www.usenix.org/conference/osdi21/presentation/lai.
[23] H. Chen, J. Ding, E.W. Tramel, S. Wu, A.K. Sahu, S. Avestimehr, T. Zhang, Self-aware personalized federated learning, in: Advances in Neural Information Processing Systems, 2022, pp. 20675–20688.
[24] P. Kairouz, H.B. McMahan, B. Avent, A. Bellet, M. Bennis, A.N. Bhagoji, K. Bonawitz, Z. Charles, G. Cormode, R. Cummings, et al., Advances and open problems in federated learning, Found. Trends® Mach. Learn. 14 (1–2) (2021) 1–210.
[25] B. Luo, X. Li, S. Wang, J. Huang, L. Tassiulas, Cost-effective federated learning design, in: IEEE INFOCOM 2021 - IEEE Conference on Computer Communications, 2021, pp. 1–10.
[26] A.H. Mohamed, N.R.G. Assumpçáo, C.A. Astudillo, A.M. de Souza, L.F. Bittencourt, L.A. Villas, Compressed client selection for efficient communication in federated learning, in: 2023 IEEE 20th Consumer Communications & Networking Conference, CCNC, 2023, pp. 508–516.
[27] F. Maciel, A.M. De Souza, L.F. Bittencourt, L.A. Villas, Resource aware client selection for federated learning in IoT scenarios, in: 2023 19th International Conference on Distributed Computing in Smart Systems and the Internet of Things, DCOSS-IoT, 2023, pp. 1–8.
[28] C.G.S. Capanema, A.M. de Souza, F.A. Silva, L.A. Villas, A.A.F. Loureiro, FedPredict: Combining global and local parameters in the prediction step of federated learning, in: 2023 19th International Conference on Distributed Computing in Smart Systems and the Internet of Things, DCOSS-IoT, 2023, pp. 17–24.
[29] S. Haykin, Neural Networks: A Comprehensive Foundation, second ed., Prentice Hall PTR, USA, 1998.
[30] D.J. Beutel, T. Topal, A. Mathur, X. Qiu, J. Fernandez-Marques, Y. Gao, L. Sani, K.H. Li, T. Parcollet, P.P.B. de Gusmão, et al., Flower: A friendly federated learning research framework, 2020, arXiv preprint arXiv:2007.14390.
[31] J. Reyes-Ortiz, D. Anguita, A. Ghio, L. Oneto, X. Parra, Human activity recognition using smartphones, 2012, http://dx.doi.org/10.24432/C54S4K.
[32] M. Malekzadeh, R.G. Clegg, A. Cavallaro, H. Haddadi, Mobile sensor data anonymization, in: Proceedings of the International Conference on Internet of Things Design and Implementation, Association for Computing Machinery, 2019, pp. 49–58.
[33] Y. Vaizman, K. Ellis, G. Lanckriet, Recognizing detailed human context in the wild from smartphones and smartwatches, IEEE Pervasive Comput. 16 (4) (2017) 62–74.
[34] Y. Vaizman, K. Ellis, G. Lanckriet, Recognizing detailed human context in the wild from smartphones and smartwatches, IEEE Pervasive Comput. 16 (4) (2017) 62–74.



Allan M. de Souza received his Ph.D. degree in computer science from University of Campinas, Brazil and University of Bern, Switzerland, in 2021. He also received his M.Sc. degree in computer science from University of Campinas, in 2016. Currently, he is professor at University of Campinas and his research interests are in the field of machine learning, distributed systems and computer networks.



Filipe Maciel is an Assistant Professor at the Federal University of Ceará at Russas with a bachelor's and master's degree in Computer Science from the State University of Ceará, having been a visiting student at Columbia University, NY. He is pursuing a Ph.D. in Computer Science from the University of Campinas, working on federated learning.



Joahannes B. D. da Costa received his Ph.D. degree in Computer Science from the University of Campinas in 2023. He received his M.Sc. degree in Computer Science from the Federal University of Para (UFPA), Brazil, in 2018. Currently, he is a postdoctoral researcher at the University of Campinas, and his research interests lie in the field of machine learning, intelligent transportation systems, vehicular clouds, resource allocation, and task scheduling.



Luiz F. Bittencourt is an Associate Professor at the University of Campinas, Brazil. Luiz was awarded with the IEEE ComSoc Latin America Young Professional Award in 2013. He acts on the organization of several conferences in the cloud and edge computing topics, and in several technical program committees. He also serves as associate editor for the IEEE Cloud Computing Magazine, for the Computers and Electrical Engineering and the Internet of Things journals, for the Journal of Network and Systems Management, and for IEEE Networking Letters. His main interests are in resource management and scheduling in cloud, edge, and fog computing.



Eduardo Cerqueira received the Ph.D. degree in informatics engineering from the University of Coimbra, Portugal, in 2008. He is currently an Associate Professor with the Faculty of Computer Engineering, Federal University of Para (UFPA), Brazil, and an Invited Researcher with the Network Research Laboratory, UCLA, USA, and the Centre for Informatics and Systems, University of Coimbra, Portugal. His publications include five edited books, five book chapters, four patents, and over than 230 articles in national/international refereed journals/conferences. His research interests include multimedia, future Internet, quality of experience, mobility, and ubiquitous computing. He has been serving as a guest editor for seven special issues of various peer-reviewed scholarly journals.



Antonio A.F. Loureiro received the B.Sc. and M.Sc. degrees (1983 and 1987) in Computer Science from Universidade Federal de Minas Gerais (UFMG), Brazil, and the Ph.D. degree in Computer Science from The University of British Columbia, Canada (1995). He is currently a Full Professor with UFMG, where he leads the research group on mobile ad hoc networks. He was the recipient of the 2015 IEEE Ad Hoc and Sensor (AHSN) Technical Achievement Award. His main research areas include ad hoc networks, mobile computing, and distributed algorithms. In the last 20 years, he has published regularly in international conferences and journals related to those areas, and have also presented keynotes and tutorials at international conferences.



Leandro A. Villas finished his doctoral degree in Computer Science at the Federal University of Minas, Brazil, in 2012. He had the doctoral thesis selected among the top 6 in the Brazilian Computer Society. He was awarded the Excellence Research Award from PARADISE Laboratory University of Ottawa. Currently, he is a CNPq researcher level 1D, Associate Professor of Computer Science at the Institute of Computing at the University of Campinas and coordinator of the Hub of Artificial Intelligence and Cognitive Architectures at Unicamp (https://hiaac.unicamp.br/en/). Dr. Villas has already published 80+ papers in international journals and 160+ in conferences. Nine of those papers received the best paper award in IEEE/ACM and SBC conferences. He was recently named among the Top 1% Scientists with the most significant impact in the world, according to a study by Stanford University/Plos Biology.